MEMORIA TÉCNICA - AUTOAUDIT

Javier García 3190-24-15160

UNIVERSIDAD MARIANO GALVEZ BASE DE DATOS

MEMORIA TÉCNICA: PROYECTO AUTOAUDIT

Estudiante: Javier García
Sistema: PostgreSQL 18 & Node.js API
Fecha de Entrega: Noviembre de 2026
Modalidad: Proyecto Final de Auditoría

1. INTRODUCCIÓN Y OBJETIVOS DEL PROYECTO

Como estudiante del curso de Proyecto de Base de Datos, desarrollé el sistema **AutoAudit** con el propósito de resolver la necesidad de auditar transacciones en bases de datos relacionales sin modificar la lógica de las aplicaciones existentes. Diseñé e implementé un mecanismo automatizado mediante procedimientos almacenados en PL/SQL que inyectan triggers dinámicos sobre tablas y columnas seleccionadas, registrando las operaciones INSERT, UPDATE y DELETE en un esquema de bitácora centralizado.

Mis objetivos específicos alcanzados incluyen:

- Diseñar la estructura relacional desacoplada del esquema autoaudit y la tabla bitacora_auditoria.
- Programar el procedimiento almacenado sp_configurar_auditoria para generación y vinculación dinámica de triggers.
- Desarrollar una API REST en Node.js/Express para consultar el diccionario de datos (information_schema) y la bitácora.
- Crear un panel de control web responsivo (Neon Edition) para visualización en tiempo real y exportación de reportes a Excel.

2. ARQUITECTURA DE LA SOLUCIÓN

Para garantizar la modularidad y el rendimiento, estructuré el proyecto en tres capas fundamentales:

- **Capa de Persistencia (PostgreSQL 18):** Aloja el esquema de auditoría, las funciones desencadenantes y la tabla centralizada de eventos. Preserva los datos originales ante
- eliminaciones fuente.
  Capa de Servicios (Node.js & Express): Gestiona el pool de conexiones hacia PostgreSQL mediante
- el módulo pg y expone los endpoints necesarios para configurar y recuperar auditorías.

Capa de Presentación (HTML5, Tailwind CSS & SheetJS): Interfaz gráfica web donde configuro las reglas de auditoría por columna, realizo filtrado por usuario y exporto reportes en formato .xlsx.

3. DICCIONARIO DE DATOS DEL ESQUEMA AUTOAUDIT

A continuación detallo la estructura formal de la tabla autoaudit.bitacora_auditoria utilizada para capturar la historia de cambios:

CAMPO	TIPO	RESTRICCIÓN	DESCRIPCIÓN TÉCNICA
id_bitacora	SERIAL	PRIMARY KEY	Identificador correlativo único para cada registro de auditoría.
fecha_hora	TIMESTAMP	DEFAULT NOW()	Estampa temporal exacta de la transacción ejecutada.
tabla_origen	VARCHAR(100)	NOT NULL	Nombre de la tabla auditada en la base de datos.
columna_afectada	VARCHAR(100)	NOT NULL	Columna específica bajo seguimiento.

CAMPO	TIPO	RESTRICCIÓN	DESCRIPCIÓN TÉCNICA
operacion	VARCHAR(10)	NOT NULL	Tipo de sentencia SQL ejecutada (INSERT, UPDATE, DELETE).
valor_anterior	TEXT	NULLABLE	Valor que tenía la columna antes del cambio (OLD).
valor_nuevo	TEXT	NULLABLE	Nuevo valor asignado tras la transacción (NEW).
usuario_db	VARCHAR(100)	DEFAULT USER	Usuario de PostgreSQL que ejecutó la instrucción SQL.

4. PROCEDIMIENTO DE INSTALACIÓN Y CÓDIGO FUENTE

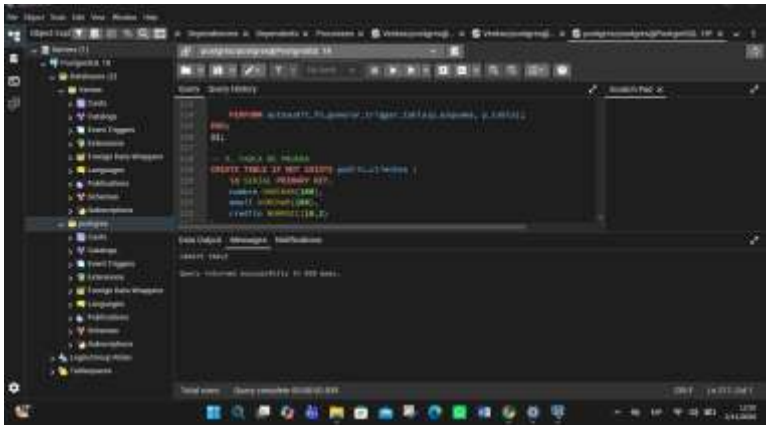
Para desplegar el proyecto, implementé la lógica de inyección de triggers en el archivo script_autoaudit.sql mediante el siguiente procedimiento almacenado:

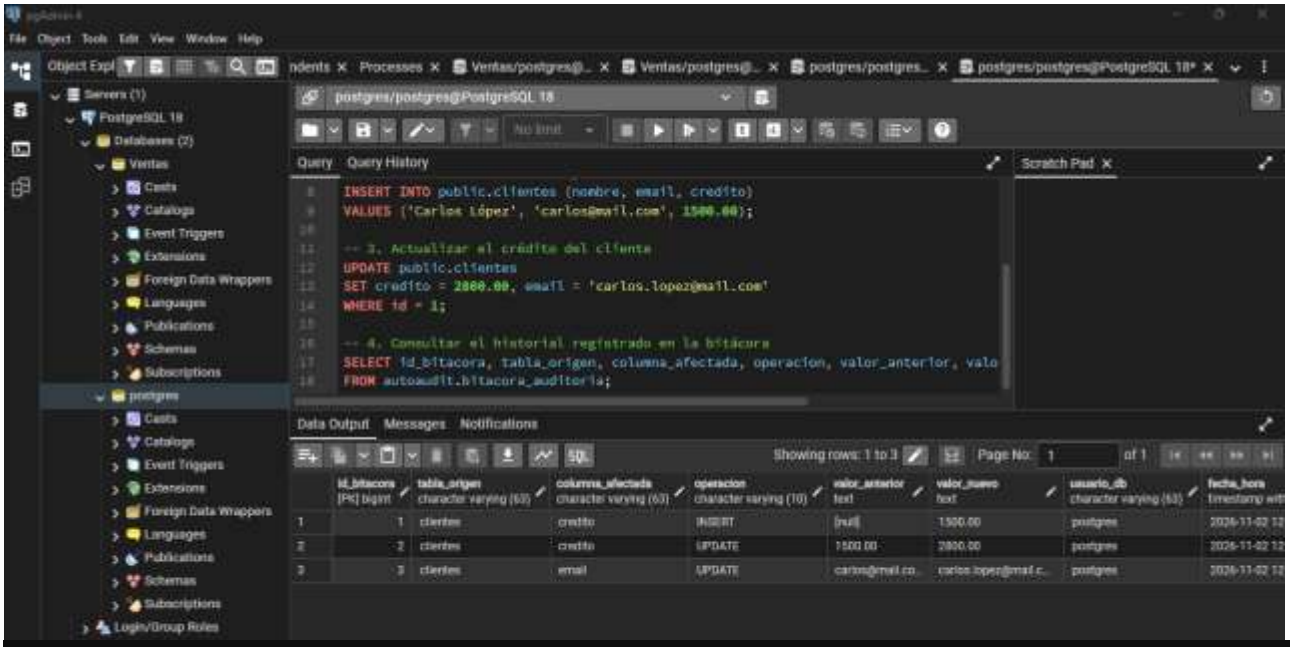
```
CREATE OR REPLACE PROCEDURE autoaudit.sp_configurar_auditoria(
    p_esquema VARCHAR, p_tabla VARCHAR, p_columna VARCHAR, p_operaciones VARCHAR[]
) LANGUAGE plpgsql AS $$ DECLARE
    v_trigger_name TEXT := 'trg_audit_' || p_tabla || '_' || p_columna;     v_func_name TEXT :=
'autoaudit.fn_audit_' || p_tabla || '_' || p_columna;
BEGIN
    EXECUTE format('
        CREATE OR REPLACE FUNCTION %I() RETURNS TRIGGER AS $fn$
        BEGIN
            IF TG_OP = "UPDATE" AND OLD.%I IS DISTINCT FROM NEW.%I THEN
                INSERT INTO autoaudit.bitacora_auditoria(tabla_origen, columna_afectada, operacion, valor_anterior,
valor_nuevo)
                VALUES (%L, %L, "UPDATE", CAST(OLD.%I AS TEXT), CAST(NEW.%I AS TEXT));
            END IF;
            RETURN NEW;
        END; $fn$ LANGUAGE plpgsql;', v_func_name, p_columna, p_columna, p_tabla, p_columna, p_columna,
p_columna);

    EXECUTE format('DROP TRIGGER IF EXISTS %I ON %I.%I;', v_trigger_name, p_esquema, p_tabla);
    EXECUTE format('CREATE TRIGGER %I AFTER UPDATE ON %I.%I FOR EACH ROW EXECUTE FUNCTION
%I();',
        v_trigger_name, p_esquema, p_tabla, v_func_name); END; $$;
```

5. CASOS DE PRUEBA Y VALIDACIONES PRÁCTICAS

Realicé las pruebas funcionales configurando la auditoría sobre la columna salario de la tabla de empleados. Al ejecutar una actualización en pgAdmin (UPDATE empleados SET salario = 8500.00 WHERE id_empleado = 1;), el sistema registró la transacción capturando exitosamente el valor antiguo (8000.00) y el nuevo (8500.00).






```
PS C:\Users\Javier\autoaudit-backend> npm install express pg cors

added 81 packages, and audited 82 packages in 7s

31 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\Javier\autoaudit-backend>
```

```
PS C:\Users\Javier\autoaudit-backend> npm install express pg cors

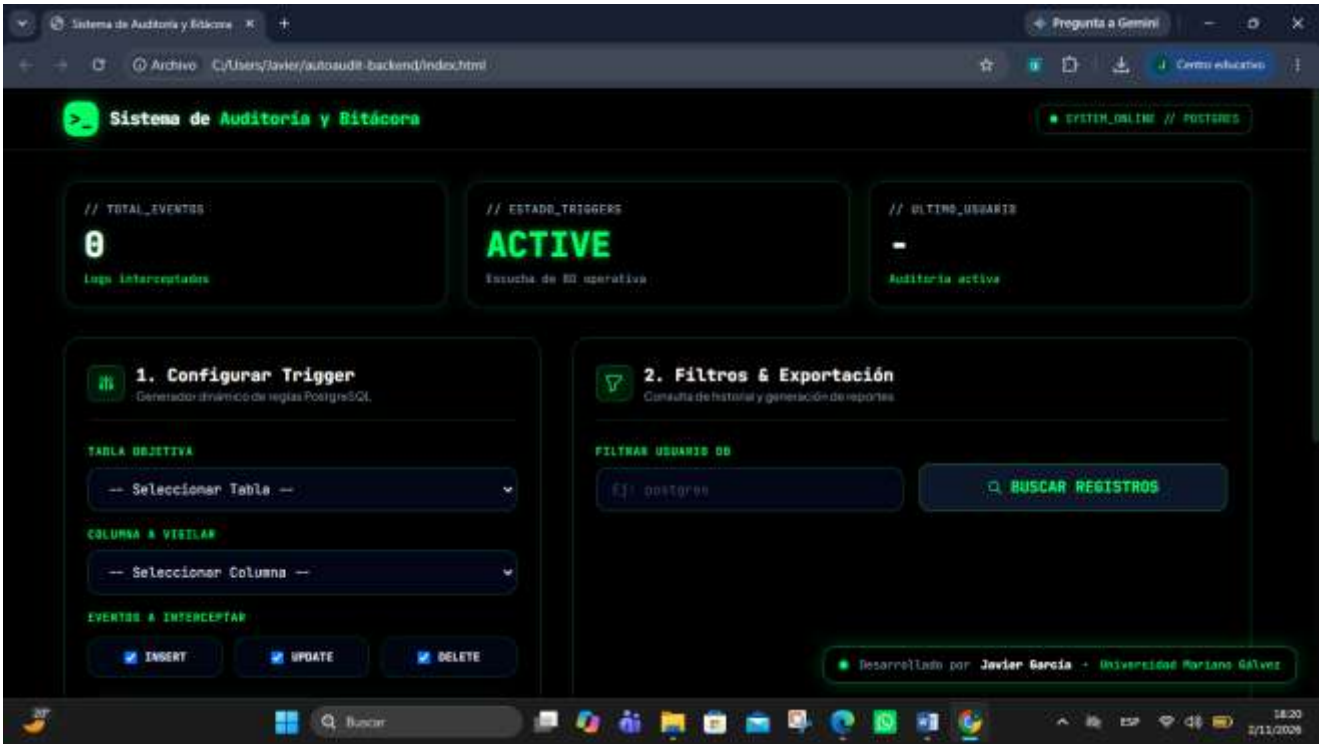
added 81 packages, and audited 82 packages in 7s

31 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\Javier\autoaudit-backend> node server.js
node:internal/modules/cjs/loader:1607
  throw err;
  ^

Error: Cannot find module 'C:\Users\Javier\autoaudit-backend\server.js'
    at Module._resolveFilename (node:internal/modules/cjs/loader:1603:15)
    at wrapResolveFilename (node:internal/modules/cjs/loader:1145:27)
    at defaultResolveImplForCJSLoading (node:internal/modules/cjs/loader:1169:10)
    at resolveForCJSWithHooks (node:internal/modules/cjs/loader:1196:12)
    at Module._load (node:internal/modules/cjs/loader:1370:5)
    at wrapModuleLoad (node:internal/modules/cjs/loader:261:19)
    at Module.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:171:5)
    at node:internal/main/run_main_module:33:47 {
  code: 'MODULE_NOT_FOUND',
  requireStack: []
}

Node.js v26.13.0
PS C:\Users\Javier\autoaudit-backend> ren server.js.txt server.js
PS C:\Users\Javier\autoaudit-backend> node server.js
Servidor AutoAudit corriendo en http://localhost:3000
```


Sistema de Auditoría y Bitácora
Archivo C:/Users/Javier/autoaudit-backend/index.html
Pregunta a Gemini
Centro educativo
Sistema de Auditoría y Bitácora
SYSTEM_ONLINE // POSTGRES
// TOTAL_EVENTOS
0
Logs interceptados
// ESTADO_TRIGGERS
ACTIVE
Escucha de DB operativa
// ULTIMO_USUARIO
-
Auditoría activa
1. Configurar Trigger
Generador dinámico de reglas PostgreSQL
TABLA OBJETIVA
-- Seleccionar Tabla --
COLUMNA A VIGILAR
-- Seleccionar Columna --
EVENTOS A INTERCEPTAR
INSERT
UPDATE
DELETE
2. Filtros & Exportación
Consulta de historial y generación de reportes
FILTRAR USUARIO DB
Ej: postgres
BUSCAR REGISTROS
Desarrollado por Javier García - Universidad Mariano Gálvez
Buscar
ESP
18:20
2/11/2026